

Relatório Técnico de Software: Sistema de Aquisição de Dados para Manutenção Preventiva de Motores

Felipe Augusto Ramos Diniz

6 de julho de 2026

1 Introdução

Este documento detalha a arquitetura de software desenvolvida para o projeto de monitoramento remoto de estações elevatórias de água. O sistema baseia-se em um microcontrolador ESP32 integrado a um modem LTE/GPRS, responsável por coletar dados de vibração, pressão, temperatura e grandezas elétricas, transmitindo-os via protocolo MQTT para um fluxo de tratamento de dados no Node-RED.

2 Estrutura e Arquivos do Projeto

O firmware do dispositivo de borda (ESP32) é modularizado em quatro arquivos principais (C/C++), complementados pelo fluxo visual de retaguarda (JSON).

2.1 SIFE_LIB.h - Sistema de Fornecimento de Energia

Responsável pelo gerenciamento autônomo de energia, implementando algoritmos de carga (CC/CV), monitoramento via sensores INA219 e controle do modo de baixo consumo (*Deep Sleep*).

- **SIFE_Setup():** Inicializa os sensores INA219, verifica os motivos de *reset* do ESP32 para determinar o Estado de Carga (SoC) inicial dinamicamente via OCV (Tensão de Circuito Aberto) ou mantendo dados da RTC, e configura o PWM.
- **iniciarDeepSleep():** Avalia a tensão atual e configura as fontes de despertar (*wake-up*). Pode adormecer por tempo programado (bateria saudável) ou indefinidamente até o retorno da rede AC via pino externo (bateria crítica), desligando também o modem.
- **ControleMosfet():** Aciona logicamente o MOSFET para isolamento ou conexão da bateria ao circuito de carga.
- **Monitora()** e **Pot_dig():** Executam a leitura contínua de corrente/tensão e atuam sobre a malha de realimentação PWM para garantir os estados corretos de carregamento (*Precharge*, Corrente Constante e Tensão Constante).

- `ContaCoulomb()`: Integra a corrente no tempo (método *Coulomb Counting*) para estimar o SoC real da bateria.
- `Gerenciamento_Carga()` e `comandos()`: Máquina de estados principal que orquestra a transição entre operação via rede, modo de bateria e rotinas de recarga.

2.2 `Connectar_Desconectar.h` - Conectividade e Criptografia

Gerencia a abstração de rede através do modem A7670 (TinyGsm), as conexões MQTT (PubSubClient) e a camada de segurança com criptografia AES.

- `sleepModem()` e `wakeModem()`: Alternam o modem para modos de baixo consumo via pinos CSCLK e DTR, otimizando o uso da bateria e acelerando reconexões de rede entre os ciclos de leitura.
- `conectarRedeEbroker()`: Realiza o processo de *boot* do modem, registro na rede celular (APN) e conexão ao *broker* MQTT. Implementa tentativas automáticas de reconexão (*retries*) em caso de falha de *socket*.
- `desconectarRede()`: Encerra graciosamente o cliente MQTT e comanda o modo suspensão do modem.
- `getVibracao()` e `enviarDadosAcelerometro()`: Fragmentam (em *chunks*) os grandes vetores de dados de vibração (512 amostras), aplicam a criptografia AES-GCM e os publicam em tópicos MQTT particionados, contornando os limites de *buffer*.
- `getMedida()` e `getEnergiaSife()`: Montam os objetos JSON para variáveis estáticas (temperatura, pressão, energia) e os criptografam antes do envio.

2.3 `backup.h` - Sistema de Persistência Offline

Garante a integridade do monitoramento em cenários de indisponibilidade de rede, salvando dados cruciais na memória RTC do ESP32.

- `Backup_Begin()` e `Backup_AtualizarCiclo()`: Calculam e atualizam contadores lógicos para determinar o momento exato em que um registro torna-se obrigatório (ex: intervalos de 6 horas).
- `Backup_ArmazenarSeNecessario()`: Chamada quando há falha de envio num ciclo crítico. Reduz a resolução geométrica da vibração para caber na memória RTC e salva estruturalmente a leitura completa.
- `Backup_EnviarPendentes()`: Executada após uma conexão bem-sucedida, varre a RTC em busca de pacotes não enviados, transmite-os com marcação de *backup* e, em caso de sucesso, limpa o *buffer*.

2.4 `principal.ino` - Orquestrador (*Main Loop*)

Agrega todas as bibliotecas e sincroniza o sequenciamento de temporização e amostragens.

- `lerTemperaturaFiltrada()`: Realiza múltiplas leituras via barramento SPI e aplica um filtro de mediana ordenado (método bolha iterativo) para mitigar ruídos do termopar.

- **collectSensorSamples():** Faz varreduras síncronas nos acelerômetros (I2C) enchendo os *buffers* primários.
- **processarLeituraEnvio():** Coordena a etapa vital do código: lê sensores, avalia conexões, envia pacotes MQTT em ordem estruturada, lida com retransmissões falhas e aciona as funções de *backup* quando a rede se mostra inoperante.
- **setup() e loop():** Configuram barramentos, registradores do *Watchdog Timer* (WDT) e definem a máquina de estados de envio temporizado versus rotinas de *Deep Sleep*.

2.5 flows.json - Retaguarda (Node-RED)

Responsável pela ingestão, decodificação, processamento matemático e armazenamento.

- **Criptografia e Roteamento (*Filtro RT/BKP*):** Descriptografa a carga AES-GCM em tempo real. Separa fluxos normais de fluxos atrasados (*backups*).
- **Reassembler (S1, S2, S3):** Reconstrói os 8 fragmentos de vibração provenientes do microcontrolador, garantindo a atomicidade do sinal amostrado (512 pontos totais).
- **FFT Nativa:** Aplica o algoritmo da Transformada Rápida de Fourier (Zero-Padding, cálculo de fase e magnitude) diretamente no Node-RED, extraindo a frequência de pico principal e sua respectiva magnitude por eixo para cada sensor.
- **Prepara MongoDB:** Formata os documentos JSON com carimbos de tempo e hierarquia de *measurements* apropriados para o banco de dados documental.

3 Fluxos de Operação e Contingência

O comportamento do firmware altera-se dinamicamente conforme os eventos do ambiente:

1. **Operação Normal:** O ESP32 desperta (via *timer*), as variáveis lidas são estáveis, a conexão 4G ocorre sem falhas TCP. Os dados são despachados, e o sistema agenda um retorno em 180 segundos (ou limite programado) via *Deep Sleep*, poupando energia.
2. **Falha de Conectividade (Rede/Broker):** O modem realiza até 3 tentativas de conexão primária. Falhando, o *Watchdog* evita o travamento e o ciclo aborta. A função de contingência avalia se o ciclo é um “marcador histórico” obrigatório. Se sim, comprime os dados, aloca na memória RTC (que sobrevive ao *Deep Sleep*) e adormece. No ciclo seguinte, se houver rede, os *backups* são purgados para o MQTT primeiro, seguidos da leitura atual.
3. **Queda de Energia (AC OFF):** A biblioteca *SIFE_LIB* identifica tensões abaixo do limiar na fonte e muda a *flag* para **FonteOFF**. Isso aciona de imediato o isolamento para proteger circuitos secundários. O ciclo de transmissão corrente é concluído, porém o temporizador de sono passa a ser regido pelo limite extremo da bateria. Ao invés do ciclo usual, o ESP32 desliga ativamente o rádio celular (*offModem()*) e configura o *wake-up ext0* (pino de hardware). O sistema entra em hibernação total e só religará quando a tensão AC for restabelecida no pino de controle.

4 Migração para o Ambiente de Produção (SVI)

A atual iteração do software opera em uma malha de testes, dependente do servidor público `broker.hivemq.com` na porta 1883. Para a adequação definitiva à infraestrutura de TI e Segurança da COPASA (SVI), é necessário aplicar as seguintes modificações estruturais em `Connectar_Desconectar.h`:

- **Endereçamento e Porta:** Substituição da constante `MQTT_SERVER` pelo IP fixo interno estabelecido pela companhia e transição para porta autenticada.
- **Autenticação de Cliente:** A rotina de tentativa de conexão (`client.connect(clientId.c_str())`) deverá receber extensões de credenciais. A chamada passará a ser obrigatoriamente `client.connect(clientId, mqttUser, mqttPassword)`.
- **APN Privada:** O direcionamento atual de GPRS (`zap.vivo.com.br`) deverá apontar para a APN dedicada (*Access Point Name* corporativo) que garante encapsulamento na VPN da empresa.